

CrewAI Multi-Agent Code Generation System

Automated Software Development via Multi-Agent AI

Author	M Aswani
Domain	Data Science / AI
Technologies	Python • CrewAI • Ollama (LLaMA 3.1) • LiteLLM

Project Overview

This project presents a multi-agent artificial intelligence system designed to automate the software development workflow. Instead of relying on a single model, the system distributes responsibilities across multiple intelligent agents, each assigned a specific role.

These agents collaborate to generate, debug, and validate Python code — mimicking how real-world software teams operate. The system is powered by a local large language model, ensuring privacy, faster execution, and zero API cost.

Objectives

- Explore how multiple AI agents can collaborate to solve problems efficiently.
- Build a structured pipeline where tasks are divided logically to improve output quality and reliability.
- Enable local execution of AI models, removing dependency on cloud services.
- Make the system accessible and practical for learners and developers.

System Architecture

The architecture follows a sequential workflow where each agent contributes to a specific stage of the process. The user provides an input problem, which is then passed through multiple agents in a structured manner.

This layered approach ensures better accuracy compared to single-step generation. Each stage acts as a quality checkpoint, improving the final result.

Developer	Code Generator	Writes clean and functional Python code based on the given problem statement.
QA Engineer	Debugging Specialist	Analyzes generated code, identifies errors, and applies corrections.
Reviewer	Validation Expert	Performs final logic validation and approves the output for reliability.

LLM Integration

The project uses **Ollama** to run a local version of a large language model (LLaMA 3.1). This allows the system to generate responses without relying on external APIs, improving speed and ensuring full data privacy.

LiteLLM serves as an alternative wrapper, enabling compatibility between different models and frameworks — making the system adaptable to various deployment environments.

Key Features

- Multi-agent collaboration with clearly defined agent responsibilities.
- Modular structure enabling easy extension and customization.
- Local LLM execution — eliminates API costs and provides execution control.
- Workflow design closely mirrors real-world software development practices.

Use Cases

- Automated coding assistant — generates and validates Python code end-to-end.
- Debugging tool — identifies logic errors and applies structured corrections.
- Educational platform — helps students understand AI workflow architectures.
- Interview showcase — demonstrates knowledge of agent-based system design.

Advantages & Limitations

Advantages	Limitations
• Cost-efficient — no paid APIs required • Faster execution via local processing • Clear role separation improves maintainability • Scalable — new agents can be added easily	• Sequential workflow may increase execution time • Output quality depends on the underlying LLM • No long-term memory or learning capabilities

Future Improvements

- Introduce parallel task execution to significantly improve speed.
- Build a user interface using Streamlit or Gradio for better accessibility.
- Integrate vector databases for long-term memory and cross-task context retention.
- Add specialized agents to handle more complex and domain-specific workflows.

Conclusion

This project highlights the potential of multi-agent AI systems in automating structured software development workflows. By combining CrewAI with a locally hosted language model, it delivers a practical, cost-effective, and scalable solution for code generation and validation.

It serves as a strong foundation for building advanced AI-driven applications and demonstrates how thoughtful system design can significantly enhance the capabilities of individual language models.